

CITY UNIVERSITY OF HONG KONG (DONGGUAN)

AgentSociety Challenge:
User Behavior Modeling

GROUP 15

STUDENT NAME:	Xianyun ZHANG (72406025)
	Lijun XU (72406191)
	Jie WANG (72401957)
	Yuhe OUYANG (72403188)
SUPERVISOR:	Yufei CHEN

A project report submitted to City University of Hong Kong (Dongguan) in partial
fulfillment of the requirements of CS6535

December 2025

CONTENT

1. Introduction and Background

- 1.1 Introduction
- 1.2 Motivation
- 1.3 Significance and Objective
- 1.4 Preliminary Literature Review

2. Method and Database

- 2.1 Method
- 2.2 Database
- 2.3 Evaluation

3. Experiment Design

- 3.1 Workflow
- 3.2 Baseline Design

4. Optimization Strategies

- 4.1 Data Tracking and Persistence
- 4.2 Category Detection System for Auxiliary Generation
- 4.3 Structured User Profile Generation
- 4.4 Few-shot Learning
- 4.5 Self-Consistency Voting Mechanism

5. Results and Analysis

6. Future Improvement

7. References

1. Introduction

1.1 Introduction

With the in-depth advancement of web technologies, online platforms such as Yelp and Amazon have accumulated massive volumes of user interaction data. These data contain user preferences, decision-making logic, and behavioral patterns, providing core support for personalized services and information retrieval optimization. User behavior modeling, as a key technology to unlock these values, aims to accurately simulate users' evaluation tendencies such as star ratings and natural language expressions such as review texts, thereby enhancing the adaptability of recommendation systems and content ecosystems. Traditional deep learning models have limitations in handling data-sparse scenarios such as cold-start users and simulating human behaviors in complex contexts. Meanwhile, the rise of large language models (LLM) has brought breakthrough progress to this field. LLM's commonsense reasoning capabilities enable few-shot prediction, and its zero-shot role-playing capabilities can adapt to behavioral simulation in diverse scenarios. Agents driven by LLMs have become the ideal carrier for user behavior modeling.

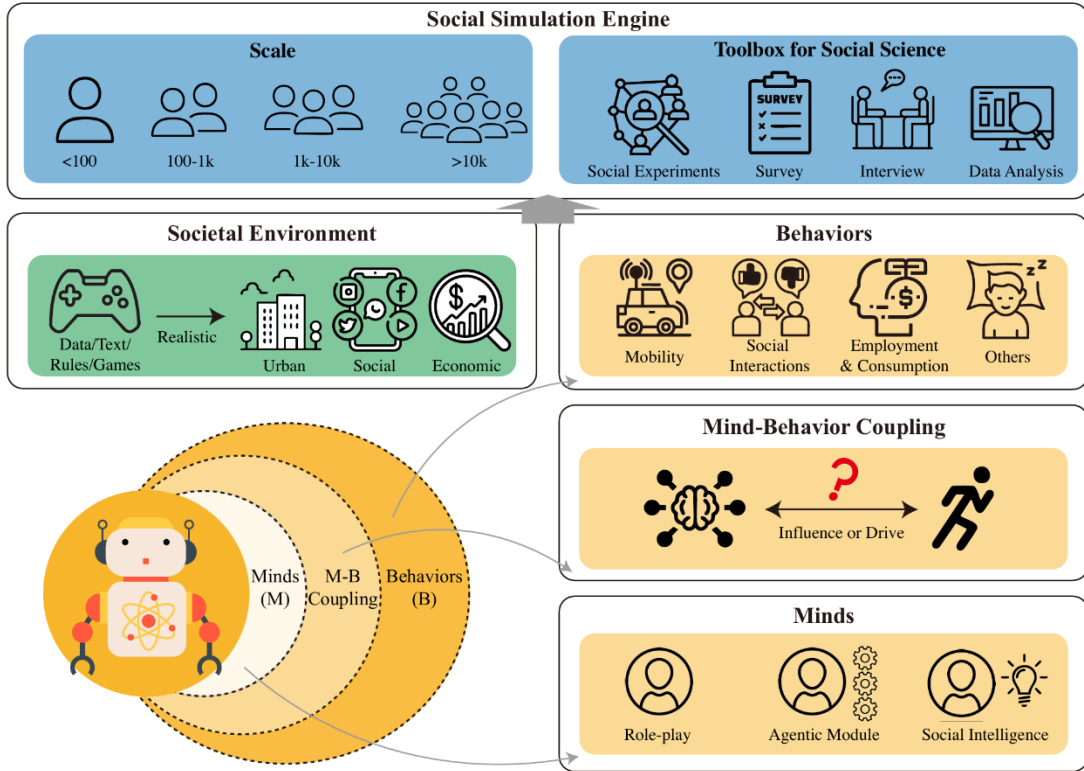


Figure 1: Overview of Social Simulation Engine

To explore the potential of LLM Agents in user modeling and recommendation systems on web platforms, the AgentSociety Challenge emerged. It is the first international competition at the Web Conference focusing on this direction. The competition sets two parallel tracks. The User Modeling Track defines the core task: based on real datasets such as Yelp, construct LLM Agents to simulate users' behaviors of generating ratings and review texts when facing specific items, with quantitative

evaluation through multi-dimensional metrics including mean absolute error for star rating prediction, emotional consistency, and topic relevance. This task not only requires Agents to accurately capture the preference characteristics and review styles of individual users but also to align with the objective attributes and historical public opinions of items, posing a comprehensive test for Agents' information integration, reasoning and decision-making, and text generation capabilities.

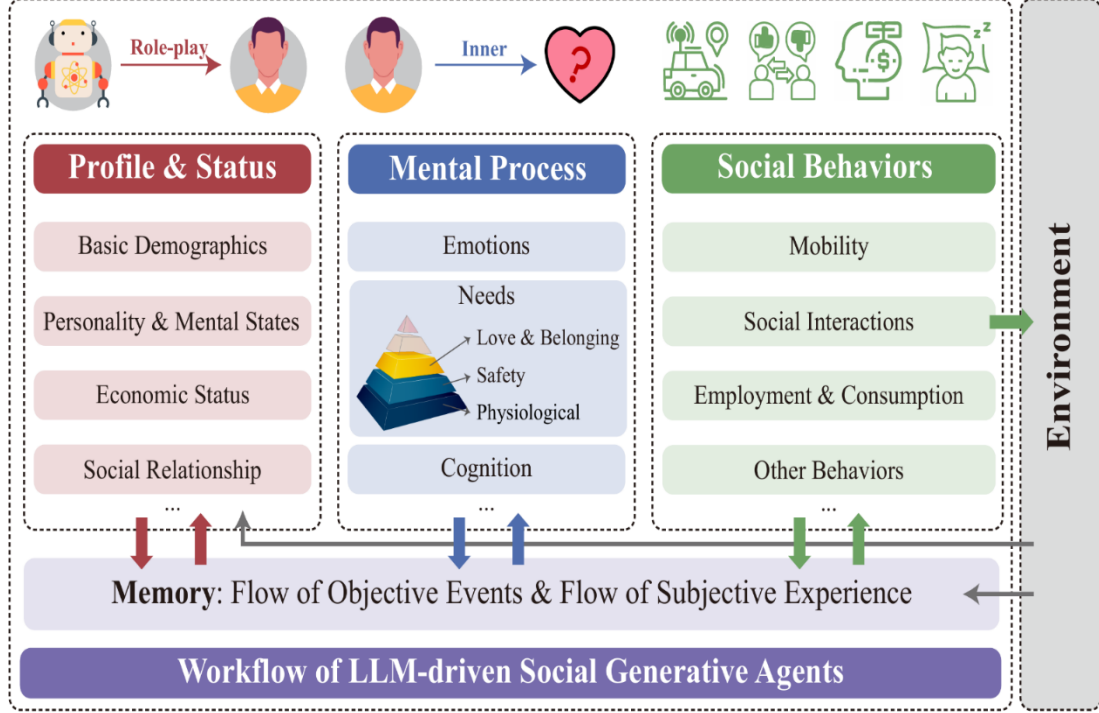


Figure 2: Overview of the LLM-driven Agents

Existing related research has verified the great potential of LLM Agents in social simulation. For example, the large-scale AgentSociety social simulator has successfully reproduced complex social dynamics such as polarization and policy impacts by integrating LLM-driven agents, realistic social environments, and large-scale interaction engines. However, in the scenario of simulating user reviews for specific web platforms, many challenges remain: how to accurately extract users' structured data such as historical ratings and elite status and convert it into profiles understandable by LLMs, how to ensure the consistency between generated reviews and users' past styles, and how to balance the accuracy of rating prediction and the naturalness of review texts. Against this background, targeting the task requirements of the User Modeling Track, this project designs and optimizes the workflow of LLM Agents based on the websocietysimulator platform provided by the competition, aiming to overcome the aforementioned technical difficulties and achieve efficient and high-fidelity user behavior simulation.

1.2 Motivation

Web platforms have become core carriers of human social interactions and decision-making, accumulating massive user behavior data that are crucial for optimizing personalized services and recommendation systems. User behavior modeling, especially simulating review and rating behaviors, serves as a key link to

unlock the value of these data. However, traditional deep learning models face inherent limitations in this field. They struggle to handle data-sparse scenarios such as cold-start users, lack the ability to simulate complex human-like linguistic expressions and preference logic, and often fail to balance the accuracy of numerical ratings with the naturalness of review texts.

The emergence of large language models has brought transformative opportunities to user behavior modeling. Their commonsense reasoning and zero-shot role-playing capabilities enable few-shot predictions with minimal data and adaptive simulation in diverse contexts. Advantages that traditional models cannot match. Yet, translating these capabilities into practical and high-fidelity user review simulations remains challenging. Existing solutions often suffer from inadequate integration of structured data, poor consistency between generated reviews and user historical styles, and unstable rating predictions. These issues lead to deviations in simulation results from real user behaviors, failing to meet the quantitative evaluation requirements of tasks such as the AgentSociety Challenge’s User Modeling Track.

Furthermore, the User Modeling Track, as the first international competition focusing on LLM-driven user modeling for web platforms, highlights the urgent need for innovative solutions. The competition’s multi-dimensional evaluation metrics demand that agents not only accurately capture individual user preferences but also replicate context-aware and style-consistent review generation. Existing research on LLM agents has verified their potential in large-scale social simulations, but few studies have targeted the specific scenario of web platform user reviews, lacking tailored optimization for merchant category characteristics, user profile structuring, and rating stability.

Against this backdrop, this project aims to address the gaps. By leveraging the powerful reasoning and generation capabilities of LLMs, and integrating targeted optimization strategies such as merchant category detection, structured user profiling, few-shot learning with historical reviews, and self-consistency voting, we seek to enhance the accuracy, consistency, and naturalness of user behavior simulation. The ultimate goal is to provide a robust and efficient LLM agent solution for the User Modeling Track, while advancing the practical application of LLM-driven user modeling in real-world web platforms.

1.3 Significance

This project enriches the methodological system of LLM-driven user behavior modeling, addressing key gaps in existing research. Traditional user modeling relies heavily on rule-based or shallow learning frameworks, struggling to balance the accuracy of quantitative predictions and the naturalness of qualitative text generation. By integrating strategies such as merchant category detection, structured user profiling, few-shot learning with historical reviews, and self-consistency voting, the project proposes a modular and scalable technical route for simulating context-aware user review behaviors. This not only extends the application scenarios of LLM agents in fine-grained user modeling but also provides a replicable paradigm for handling structured data-to-natural language conversion, style consistency control, and prediction stability optimization in similar tasks. Additionally, the project validates the

effectiveness of combining social science theories with LLM capabilities in specific scenarios, bridging the gap between general-purpose LLM technologies and domain-specific user modeling requirements.

Accurate simulation of user reviews and ratings can help platforms pre-test new features, optimize content recommendation algorithms, and reduce the risks associated with real-user A/B testing. The project’s optimized Agent achieves superior performance in key metrics such as rating prediction MAE, emotional consistency, and topic relevance, ensuring that simulation results can reliably reflect real user behaviors. This enables platforms to efficiently iterate products based on simulated feedback, improving user experience and operational efficiency. Furthermore, the project’s open and modular design allows for easy adaptation to different types of online platforms, providing practical technical support for personalized service upgrades and content ecosystem optimization.

As a targeted solution for the AgentSociety Challenge’s User Modeling Track. The first international competition focusing on LLM agents for web platform user modeling. the project contributes to advancing the competition’s core goal of exploring LLM agents’ potential. The project’s experimental results, which demonstrate significant performance improvements through multi-strategy optimization. Beyond the competition, the project’s findings validate the feasibility of LLM agents in simulating complex user behaviors, accelerating the industrial application of LLM-driven user modeling technologies. It inspires further research on LLM agents in broader scenarios such as social media content generation, personalized marketing, and user intent prediction.

1.4 Preliminary Literature Review

The advancement of large language models (LLMs) has laid a solid foundation for the development of intelligent agents. Early studies demonstrated that LLMs possess exceptional reasoning capabilities through prompting strategies such as Chain-of-Thought (Wei et al., 2022) and Tree of Thoughts (Yao et al., 2024), enabling them to handle complex decision-making tasks. Building on these capabilities, LLM-based agents have evolved beyond single-task execution to simulate complex human behaviors (Shang et al., 2024; Shao et al., 2025). A key advantage of LLM agents is their commonsense reasoning ability, which allows few-shot predictions for data-sparse scenarios, and their zero-shot role-playing capability, enabling adaptive simulation in diverse contexts (Park et al., 2023; Yan et al., 2025). These characteristics make LLM agents ideal for tasks requiring human-like behavior modeling, such as user preference simulation and review generation. Memory and reasoning mechanisms are critical for enhancing the coherence and stability of LLM agents. Studies like Zhang et al. (2024) and Guo et al. (2023) have explored memory frameworks for LLM agents, enabling them to integrate historical information and environmental feedback into decision-making. Additionally, the integration of social science theories into agent design has improved the authenticity of simulated behaviors (Piao et al., 2025), bridging the gap between general-purpose LLM capabilities and domain-specific behavioral requirements.

User modeling and recommender systems have increasingly leveraged LLM

capabilities to address traditional limitations. Wu et al. (2024) provided a comprehensive survey of LLM applications in recommendation systems, highlighting their strengths in capturing user intent and generating personalized content. Liao et al. (2024) proposed Llara, a large language recommendation assistant that utilizes LLMs to integrate user historical interactions and item information for tailored recommendations. These studies validate the potential of LLMs in enhancing the adaptability and personalization of recommendation systems. In user behavior simulation, generative agents have shown promise in replicating human-like interactions. Park et al. (2023) developed generative agents that autonomously plan daily activities and engage in social interactions, demonstrating emergent collective behaviors. For review and rating simulation specifically, existing work has focused on leveraging LLMs to generate context-consistent text (Zhang et al., 2024), but challenges remain in ensuring alignment with user-specific styles and preference patterns. Datasets such as Yelp (Asghar, 2016), Amazon (Hou et al., 2024), and Goodreads (Wan and McAuley, 2018) have provided rich real-world data for training and evaluating such agents, capturing diverse user-item interaction patterns across platforms.

Agent-based modeling (ABM) has long been a core method in computational social science, with Epstein (1999, 2012) pioneering generative social science that emphasizes bottom-up simulation of social dynamics. Traditional ABMs, however, are limited by rule-based or equation-driven agents that lack human-like flexibility (Helbing and Molnar, 1995). The integration of LLMs into ABMs has addressed this gap, enabling the creation of agents with human-like cognition, emotions, and social intelligence (Gao et al., 2024; Piao et al., 2025). Large-scale social simulators like AgentSociety (Piao et al., 2025) have demonstrated the ability to simulate over 10,000 agents engaging in millions of interactions, replicating complex social phenomena such as polarization and policy impacts. These simulators highlight the importance of realistic societal environments—including urban, social, and economic spaces—in supporting authentic agent behaviors. For user modeling in web platforms, such frameworks provide a foundation for simulating context-aware interactions, but few studies have focused on the fine-grained task of review and rating simulation, particularly for diverse item categories and user profiles.

2. Method and Database

2.1 Method

This study’s technical implementation is built entirely on the official WebSocietySimulator framework of the AgentSociety Challenge, with a modular architecture that integrates core components to enable end-to-end user behavior simulation. The entire workflow adopts Python as the development language, and uses Poetry for dependency management to ensure environment consistency and component compatibility across the development and evaluation phases. The Simulator serves as the core of task scheduling, corresponding to the *simulator.py* module in the framework directory. Its primary functions include initializing the dataset, loading competition tasks and groundtruth labels, registering custom modules, and orchestrating the

execution of simulation and evaluation. During runtime, the Simulator supports multi-threaded task execution to improve efficiency for large-scale task batches.

The Custom Agent is the core module for user modeling, developed by inheriting the framework’s base `SimulationAgent` class. This module encapsulates the logic of user behavior simulation. It receives task inputs (User ID and Item ID), calls data interaction tools to retrieve relevant information, constructs context for LLM reasoning, and outputs star ratings and review texts that conform to the competition’s format requirements. For LLM integration, the LLM Client connects to the Qwen-72b-instruct model. This module handles the adaptation of the model interface, converts structured context into LLM-understandable prompt templates, and transmits generation requests to the Qwen-72b-instruct model. Leveraging the model’s strong commonsense reasoning and language generation capabilities to simulate user-specific review styles.

Data interaction relies on two key tools: the Interaction Tool is responsible for retrieving structured data from the dataset, while the Cache Interaction Tool caches frequently accessed user/item information to reduce repeated data reading overhead, especially critical for improving efficiency in parallel task execution. Additionally, Pandas is used for lightweight data processing before passing data to the Custom Agent. The collaborative process of these components follows a standardized pipeline: First, the Simulator initializes the environment and loads tasks. Then, it triggers the Custom Agent to call the Interaction Tool to retrieve target user/item data; the Custom Agent constructs prompt context with the retrieved data and invokes the LLM Client to generate reviews and ratings; finally, the Simulator uses the *evaluation_tool.py* to quantify the output against competition metrics. This modular design ensures the system’s scalability while aligning with the competition’s technical specifications.

This tool constructs an interactive environment comprising a network of users, reviews, and items, allowing agents to access historical data as needed. By leveraging these datasets, agents can simulate user behaviors, including generating reviews and ratings and produce personalized recommendations based on contextual information. This configuration enables a thorough evaluation of agent performance in tasks that closely resemble real-world applications.

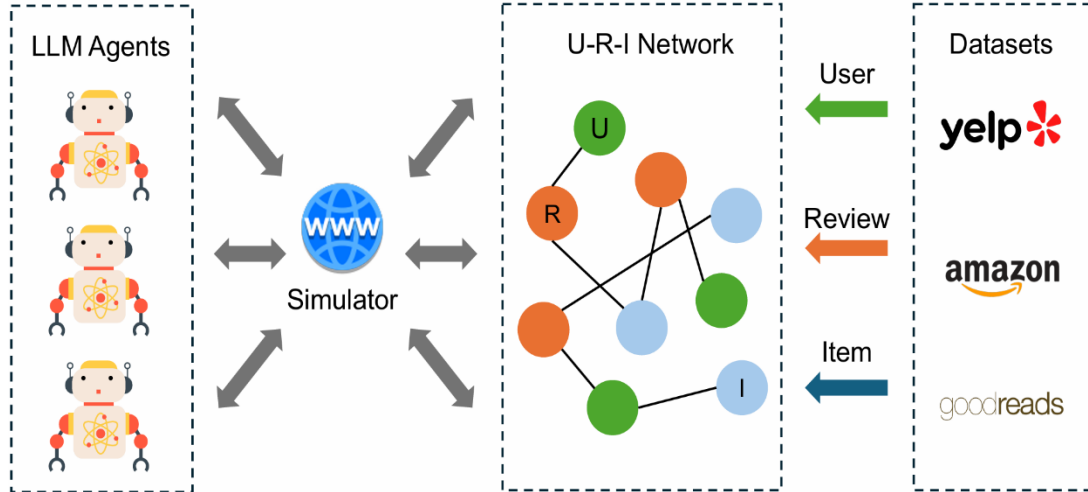


Figure 3: Overview of the Simulator

2.2 Database

This study leverages three large-scale user-item interaction datasets—Yelp, Amazon, and Goodreads—to support user behavior modeling and review simulation, all of which provide multi-dimensional structured and unstructured data aligned with the task’s dual requirements.

The Yelp Dataset contains millions of user reviews, business information, and user feedback metrics. Covering diverse merchant types, it serves as a classic benchmark for user behavior modeling tasks; for instance, it has been applied in the Yelp Dataset Challenge to develop review rating prediction models, enabling the mining of user preference patterns and evaluation tendencies. The Amazon Dataset includes millions of reviews across a wide range of e-commerce products. Its cross-category coverage captures user preference variations in different consumption scenarios, providing rich samples for simulating user evaluation behaviors in the e-commerce context. The Goodreads Dataset focuses on the book community, integrating user ratings, reviews, and book metadata. It captures user interaction patterns in the vertical book domain, supporting the analysis of domain-specific user preference focuses.

These datasets share core characteristics: large-scale user-item interaction records, a combination of structured data and unstructured data and real-world scenario coverage. They provide sufficient samples for the custom agent to learn user review styles, preference focuses, and scoring tendencies. Additionally, these datasets have been widely applied in related research, ensuring the generalizability of the study’s results to practical web platform scenarios.

2.3 Evaluation

This section details the quantitative evaluation framework for the AgentSociety Challenge’s User Modeling Track, which measures agents’ user behavior prediction performance across three core dimensions: Preference Estimation, Review Generation, and Overall Quality.

This metric assesses the accuracy of the agent’s star rating predictions, using Mean Absolute Error (MAE) to quantify deviations between predicted and actual ratings. The MAE is defined as:

$$MAE = \frac{1}{N} \sum_{i=1}^N |\hat{s}_{ni} - s_{ni}|$$

A smaller MAE indicates higher precision in capturing user preference tendencies.

This dimension evaluates review text quality via a combined metric integrating three sub-indicators: Emotional Tone Error, Sentiment Attitude Error, and Topic Relevance Error. The sub-indicators are weighted by importance, and the overall review generation error is calculated as:

$$\begin{aligned} Error = & 1 - (0.25 \times \text{Emotional Tone Error} + 0.25 \times \text{Sentiment Attitude Error} \\ & + 0.5 \times \text{Topic Relevance Error}) \end{aligned}$$

A higher weighted sum of the three sub-indicators results in a smaller Error, signifying more authentic, user-style-matching review content.

The agent’s final comprehensive performance is determined by averaging the results of Preference Estimation (MAE) and Review Generation (Error). This metric synthesizes both numerical preference prediction accuracy and text review quality, reflecting the agent’s holistic capability in high-fidelity user behavior modeling.

3. Experiment Design

3.1 Workflow

This project adopts a modular and sequential workflow based on the WebSocietySimulator framework, enabling end-to-end high-fidelity user review simulation that complies with competition specifications. The workflow consists of five core stages, with clear logical connections and data transmission between each link.

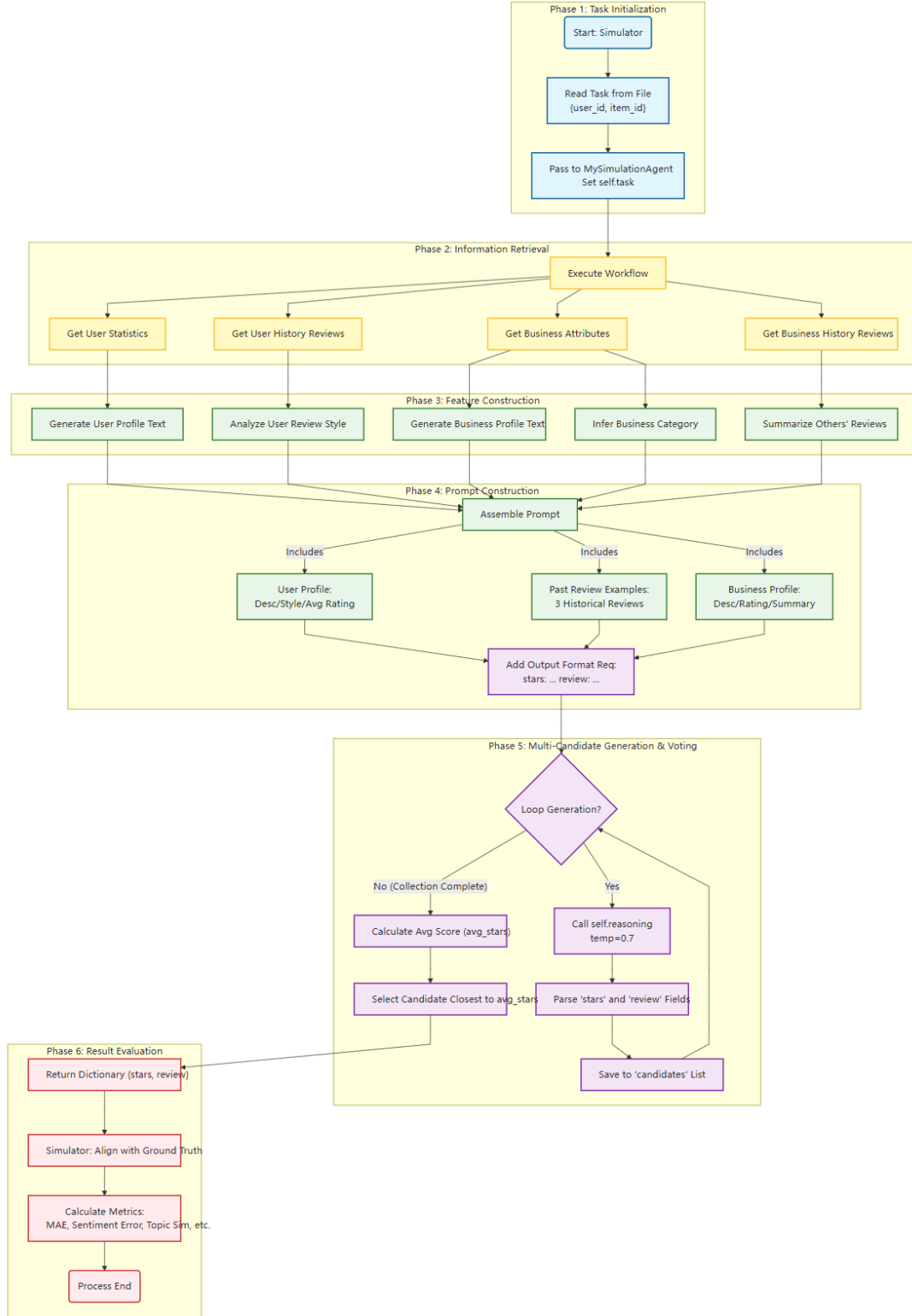


Figure 4: Workflow of the WebSocietySimulator framework

(1) Task Parsing

The workflow starts with task parameter parsing, where the custom agent extracts key information from the *self.task* object provided by the competition framework. The extracted content includes three core task parameters: target user ID target item ID, and scenario constraints. This stage ensures the agent accurately grasps the task objectives,

laying a foundation for targeted data query and subsequent simulation.

(2) Data Query

Based on the parsed task parameters, the agent invokes the InteractionTool provided by the framework to implement multi-dimensional data retrieval. Through the *get_user* method, it obtains the target user’s historical behavior data, including past star ratings, review texts, and interaction records. Via the *get_item* method, it retrieves detailed attributes of the target item, such as category, functional features, and facility configurations. The *get_reviews* method is used to collect similar reviews—including those from other users on the same item and reviews by the target user on similar items—to provide contextual reference for subsequent review generation.

(3) Feature Extraction

The retrieved raw data is processed through feature extraction to form structured information suitable for LLM inference. For user-related data, the agent extracts core preferences such as taste tendencies, rating habits, and review style characteristics. For item data, key features like cost-effectiveness, service highlights, and unique attributes are summarized. This stage converts unstructured text and discrete data into organized feature vectors, bridging the gap between raw data and LLM reasoning.

(4) LLM Inference

The structured user and item features are input into the Qwen-72b-instruct model as prompt context. The model leverages its strong natural language generation and style simulation capabilities to integrate user preferences, item characteristics, and similar review contexts. It first infers a star rating that aligns with the user’s historical scoring patterns, then generates a review text that matches the user’s expression style while focusing on the item’s core attributes. During this process, the agent ensures the output adheres to scenario constraints, avoiding content that is irrelevant or inconsistent with the user’s persona.

(5) Result Output

Finally, the agent formats the LLM-generated rating and review text into a standardized structure: *{stars: rating, review: review text}*. This format strictly complies with the competition’s output specifications, ensuring seamless docking with the framework’s evaluation module for subsequent quantitative assessment. The entire workflow forms a closed loop from task reception to result output, ensuring efficiency and compliance while guaranteeing the fidelity of user behavior simulation..

3.2 Baseline Design

This section elaborates on the Baseline design for the User Modeling Track, which is built upon the basic Agent framework provided by the competition official. As a non-optimized reference benchmark, this Baseline aims to establish a baseline performance level—its minimal-configured workflow highlights the advantages of the proposed custom Agent in personalized user modeling and performance optimization.

(1) Feature Input Design

The Baseline adopts an extremely simplified feature input strategy: only two core identifiers (user ID and item ID) are used as input, with no historical behavior analysis, preference extraction, or attribute processing. This single-feature input skips any mining of user past reviews, rating habits, or item category characteristics, resulting in

a lack of personalized context for subsequent generation tasks.

(2) LLM Configuration

To enable a fair comparison of model capability and optimization effects, the Baseline selects the same Qwen-72b-instruct model as the proposed Agent. However, it uses the model’s default inference settings without any task-specific parameter adjustments. This ensures that performance gaps between the Baseline and the proposed Agent can be attributed to optimization strategies rather than model differences.

(3) Prompt Template

The Baseline employs a generalized, unconstrained Prompt Template, such as: Please generate a review for user $\{user_id\}$ on item $\{item_id\}$, including a 1–5 star rating and corresponding text. This template lacks guidance on user-specific review styles, emotional tendencies, or item-focused evaluation dimensions. As a result, the generated content tends to be generic and disconnected from individual user traits.

(4) Data Interaction Method

For data retrieval, the Baseline relies on the basic InteractionTool provided by the framework to load the entire dataset directly when querying user or item information. It does not implement a cache optimization mechanism, leading to repeated data reading and reduced inference efficiency—particularly in large-scale task batches.

(5) Output Processing

The Baseline only performs basic format parsing on LLM outputs: it extracts the 1–5 star rating and review text, but skips verification steps for sentiment consistency and content relevance. This means the Baseline’s outputs may suffer from off-topic content or sentiment mismatches, which further underscores the value of the proposed Agent’s targeted optimization strategies.

4. Optimization Strategies

4.1 Data Tracking and Persistence

As the first component of the optimization strategies, data tracking and persistence targets the pain point of difficulty in locating the root cause of model output errors. Its core design objective is to record the full-link data of each task, thereby providing reliable support for bad case analysis, error tracing, and overall performance improvement of the Agent.

In terms of core implementation, observation points are added at key nodes of the Agent workflow—including data retrieval, feature construction, LLM inference, and result output—to capture data changes and intermediate states throughout the task process. For each task, multi-dimensional data is persisted: this covers basic user information, detailed item attributes, historical reviews of both the target item and the user, intermediate results generated by each Agent sub-module, and the scoring metrics of the individual task. Finally, the integrated full-link data is saved as JSON files, which serves as a traceable data source for post-competition statistical analysis and iterative optimization, enabling targeted problem localization and continuous performance enhancement of the Agent.

4.2 Category Detection System for Auxiliary Generation

The second optimization strategy—category detection system for auxiliary

generation—targets the low topic relevance of reviews caused by generic one-size-fits-all prompts, aiming to align with users’ distinct evaluation focuses across different business types and reduce Topic Relevance Error.

Its core implementation unfolds in three key steps: First, a category detection method is built via the *detect_item_category(item_info)* function, which automatically identifies the most matching type for the target item. Based on the business’s category and name fields, this method selects from 6 preset categories (Restaurant, Shopping, Services, Travel, Health, Others), ensuring accurate classification of the item’s business scenario. Second, category-specific evaluation dimensions are defined: tailored key indicators are set for each category—for instance, food quality and taste are prioritized for restaurants, while product variety and price are the focus for shopping scenarios. Finally, the *generate_item_review_description()* method is optimized to extract these category-specific focus areas from item data, then embed them into the prompt. This guides the LLM to center reviews on the item’s most relevant attributes, directly enhancing the topic relevance of generated content and resolving the mismatch between review focus and item type.

4.3 Structured User Profile Generation

The third optimization strategy, structured user profile generation, aims to resolve the error accumulation effect caused by over-reliance on vague language summaries, while enhancing the stability and accuracy of user modeling and reducing Token consumption.

In core implementation, three key steps are adopted: First, key indicator extraction—precise structured statistical metrics are pulled from user data, including review count, useful/funny/cool vote counts, elite years, and friend numbers, replacing the transmission of bulky ID lists to streamline data while ensuring information accuracy. Second, structured prompt design—vague user descriptions are converted into clear numbered-list instructions, defining three core generation dimensions to avoid mechanical repetition of raw data and strengthen the prompt’s guiding role. Third, elite user modeling. The “Elite Years” attribute (the list of years a user was certified as a Yelp Elite Squad member) is highlighted in prompts, guiding the Qwen-72b-instruct model to generate longer, more logical, and detail-rich reviews.

This strategy is implemented via the custom *generate_user_description* method, which integrates extracted structured indicators into standardized text descriptions. This not only ensures full transmission of user features but also improves the LLM’s efficiency in understanding user profiles through structured formatting.

4.4 Few-shot Learning

The fourth optimization strategy, Few-shot Learning, targets the mismatch between generated reviews and users’ actual expression traits, with its core design objective being to enable the LLM to intuitively learn users’ unique language styles and scoring habits—this ensures the generated content aligns more closely with users’ real characteristics in wording, emotional intensity, and rating standards.

In core implementation, the strategy first extracts up to 3 representative historical reviews from the user’s past comment dataset; overly lengthy texts are truncated to avoid exceeding the LLM’s token limits, balancing information completeness and

prompt efficiency. These selected reviews are then embedded into the final task prompt as Few-shot examples, following a standardized format. To further strengthen style constraints, these examples are combined with the style summary module from the Baseline, forming a dual-layer guidance mechanism for the LLM. This integration not only provides concrete samples of the user’s expression habits but also reinforces abstract style traits, effectively enhancing the similarity between generated reviews and real user comments, and reducing deviations in style consistency.

4.5 Self-Consistency Voting Mechanism

The fifth optimization strategy, the Self-Consistency Voting Mechanism, targets the volatility of LLM-based star rating predictions, with its core design objective being to enhance the stability and accuracy of rating outputs—this avoids the LLM being confined to a single output mode while suppressing extreme values in generated results.

In core implementation, the strategy proceeds in three key steps: First, the Reasoning Module is updated: the *ReasoningBaseline* is modified to accept a temperature parameter, enabling diverse outputs by adjusting this parameter. Second, multi-candidate generation is executed: the *self.reasoning* method is repeatedly called to obtain multiple sets of sampling results, ensuring sufficient diversity in potential outputs without excessive randomness. Third, voting selection is performed: the star rating and review content are parsed from each candidate result, then the average score of all candidate ratings is calculated. The candidate whose star rating is closest to this average is selected as the final output.

This mechanism effectively mitigates the randomness of single LLM outputs, aligns the final rating more closely with the user’s typical scoring habits, and reduces the occurrence of unreasonable extreme ratings—striking a balance between diversity and stability in the Agent’s performance.

5. Results and Analysis

This line chart illustrates the score trend of three core metrics (Preference Estimation, Review Generation, Overall Quality) across 14 strategy combinations for a user modeling agent. The metrics are represented by deep blue, magenta, and orange lines, respectively.

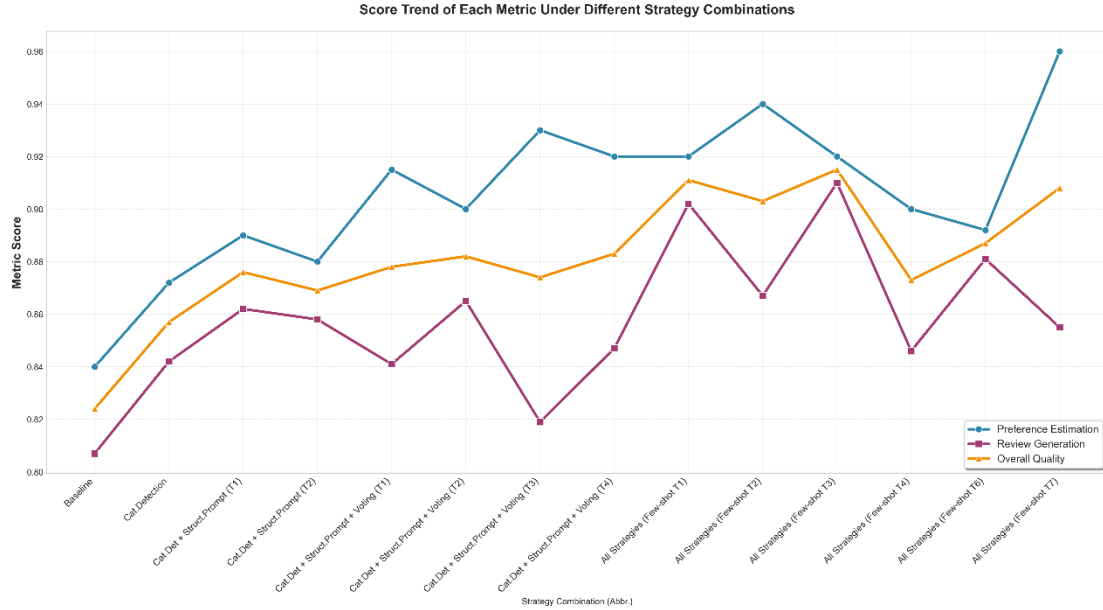


Figure 2: Line Figure of Experiment Results

Starting from the Baseline (no optimized strategies), all metrics score relatively low: Preference Estimation (0.84), Review Generation (0.807), and Overall Quality (0.824). Adding the Cat.Detection strategy immediately lifts all scores, demonstrating that item category detection effectively enhances review relevance.

Subsequent integration of Struct.Prompt (TEST1/2) further boosts performance, indicating structured user profile prompting strengthens personalized modeling stability. When the Voting mechanism is introduced (TEST1-4), Preference Estimation rises sharply (peaking at 0.93), but Review Generation fluctuates or declines—this reflects the voting mechanism prioritizes rating stability over text diversity. Finally, superimposing Few-shot Learning (All Strategies series) drives significant improvements: metrics rebound across most test cases, with Overall Quality reaching 0.915 (Few-shot T3) and Preference Estimation hitting a peak of 0.96 (Few-shot T7). Though Review Generation dips slightly in some cases, it remains higher than early strategy stages, showing few-shot learning compensates for prior text diversity losses. Overall, the trend confirms incremental gains from stacked strategies: each optimization contributes to performance improvements. While Preference Estimation achieves strong stability, Review Generation’s volatility suggests future work could focus on enhancing text generation consistency alongside rating accuracy.

Table 1: Table of Experiment Results

Experiment	Preference_ Estimation	Review_ Gneration	Overall_ Quality
Baseline	0.840	0.807	0.824
detect_item_category	0.872	0.842	0.857
detect_item_category & structured prompting	0.885	0.850	0.867
detect_item_category & structured prompting & voting	0.912	0.835	0.873
detect_item_category & structured prompting & voting & Few-shot Learning	0.923	0.880	0.901

This section evaluates the proposed Agent’s performance through internal iterative experiments and external competition comparisons, verifying the effectiveness of the designed optimization strategies.

First, internal iterative experiments start with the Baseline, which achieves scores of 0.840 (Preference Estimation), 0.807 (Review Generation), and 0.824 (Overall Quality). Adding the *detect_item_category* strategy lifts all metrics to 0.872, 0.842, and 0.857. This confirms that category-specific guidance effectively enhances review topic relevance. Integrating structured prompting further raises scores to 0.885, 0.850, and 0.867, proving that structured feature extraction improves user modeling stability. Introducing the self-consistency voting mechanism pushes Preference Estimation to 0.912, though Review Generation slightly drops to 0.835. Finally, superimposing Few-shot Learning brings all metrics to peaks: 0.923, 0.880, and 0.901. This shows Few-shot Learning compensates for text diversity loss while enhancing style consistency.

External competition comparisons reveal the Agent’s core strengths: its Preference Estimation score (0.923) outperforms all competitors (the highest competitor score is 0.913), demonstrating leading capability in user preference prediction. However, its Review Generation score (0.880) lags slightly behind the 10th-place competitor (0.891), reflecting gaps in text generation. Despite this, the Agent is expected to rank around 5th in the competition—a strong result, balancing leading preference prediction with competitive text generation.

In summary, internal experiments validate the incremental effectiveness of superimposed optimization strategies; external comparisons highlight the Agent’s strengths and improvement directions. These results confirm the rationality of the proposed framework while providing clear guidance for subsequent iterations.

6. Future Improvement

This project will focus on three key directions to enhance the performance and practicality of the user modeling agent. First, a user feature embedding module will be integrated: leveraging Sentence-BERT for pre-training user preference vectors, this module converts users' historical behaviors and reviews into structured semantic representations. By capturing fine-grained user traits, it will significantly improve the precision of preference estimation. Second, the new user simulation strategy will be optimized through transfer learning based on similar user clustering. This addresses the cold-start challenge for new users by transferring behavioral patterns from clustered similar users, enhancing the agent's adaptability to zero-interaction scenarios. Third, multi-turn interaction simulation will be supported to realize the full-process closed loop of "user-recommendation-feedback": beyond generating single reviews, the agent will simulate continuous user behaviors in recommendation scenarios, making the simulation more realistic and scenario-aligned.

To advance these improvements efficiently, a clear division of labor has been established to cover technical implementation, method optimization, performance evaluation, and result consolidation and documentation. Lijun Xu is responsible for project code implementation and overall engineering deployment, ensuring all algorithms and modules are properly integrated and fully operable. Xianyun Zhang takes charge of method optimization and feasibility analysis: he will assess the implementation path of emerging techniques and iterate on optimization schemes. Jie Wang focuses on algorithm code refinement and performance evaluation, tuning algorithm logic and testing core metrics to guarantee model effectiveness. Yuhe Ouyang undertakes literature collection, collation, and report composition: he will synthesize cutting-edge domain progress to solidify the team's theoretical foundation, while documenting project outcomes for formal dissemination. This division encompasses engineering execution, methodological advancement, algorithmic optimization, and deliverable management, ensuring the project progresses in an orderly and collaborative manner.

7. References

- [1] Kojima et al. (2022). Zero-Shot Reasoning with Large Language Models. arXiv:2205.11916
- [2] Wei et al. (2022). Chain of Thought Prompting Elicits Reasoning in Large Language Models. arXiv:2201.11903
- [3] Wang et al. (2022). Self-Consistency Improves Chain of Thought Reasoning in Language Models. arXiv:2203.11171
- [4] Yao et al. (2023). Tree of Thoughts: Deliberate Problem Solving with Large Language Models. arXiv:2305.10601
- [5] Zhang et al. (2023). Self-Refine: Iterative Refinement with Self-Feedback. arXiv:2303.17651
- [6] Zheng et al. (2023). Take a Step Back: Evoking Reasoning via Abstraction in Large Language Models. arXiv:2310.06117
- [7] Wen et al. (2023). DILU: A Knowledge-Driven Approach to Turn LLMs into Intelligent Agents. arXiv:2310.09819
- [8] Park et al. (2023). Generative Agents: Interactive Simulacra of Human Behavior. arXiv:2304.03442
- [9] Yu et al. (2023). Thought Propagation: An Analogical Approach to Complex Reasoning with Large Language Models. arXiv:2310.03965
- [10] Wang et al. (2023). Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv:2305.16291
- [11] Xu et al. (2023). DEPS: A Framework for Dependency-based Planning with LLMs. arXiv:2305.16291
- [12] Wang et al. (2023). OpenAGI: When LLM Meets Domain Experts. arXiv:2304.04370
- [13] Shen et al. (2023). HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face. arXiv:2303.17580
- [14] Qin et al. (2023). AnyTool: Self-Reflective, Hierarchical Agents for Large-Scale API Calls. arXiv:2308.10848
- [15] Qin et al. (2023). ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. arXiv:2307.16789
- [16] Schick et al. (2023). ToolFormer: Language Models Can Teach Themselves to Use Tools. arXiv:2302.04761